41. Containers and Multi-stage Docker Builds for Go

41. Containers and Multi-stage Docker Builds for Go

Phase: Phase 6 — Systems and infrastructure
Estimated time: 1–2 days
Slug: `41-containers-docker`

Learning objectives

- Write a multi-stage Dockerfile for Go
- Build static/small images
- Run as non-root
- Understand layer caching

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Multi-stage Docker builds keep Go images tiny and secure. Containers are how Go services ship.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Build stage with golang image; runtime with distroless/scratch/alpine.
- COPY go.mod/sum first for layer caching.
- CGO_ENABLED=0 for static binaries when possible.
- Run as non-root; expose minimal ports.
- Healthchecks and read-only rootfs where practical.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-containers && cd /tmp/gw-containers
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func main() {
    // Inside container this binary is often /app
    fmt.Println("listening :8080")
}
```

Run it:

```
go run .
```

Expected output:

```
listening :8080
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write a multi-stage Dockerfile sketch on paper.
2. List what NOT to COPY (secrets, .git).
3. Explain why go mod download is a separate layer.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Multi-stage pattern

```
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/app ./cmd/api

FROM gcr.io/distroless/static:nonroot
COPY --from=build /out/app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
```

Why

- Tiny attack surface
- Reproducible builds
- Same artifact from laptop to prod

Pin base image digests in serious deployments. Pass version via ldflags.

Failure modes to watch while coding

- Shipping build toolchain in prod image
- Running as root
- COPY . . before go mod download (breaks cache)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- venv in images vs static Go binaries — Go wins size often.
- Same Docker layer caching ideas.
- GIL/process model differs; still one process per container usually.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Pin base image digests for supply-chain hygiene.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Containers and Multi-stage Docker Builds for Go" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Shipping build toolchain in prod image
- Running as root
- COPY . . before go mod download (breaks cache)

Exercises

Work in `exercises/41-containers-docker/`. Read that folder's `README.md` before you code.

What you will build

Complete Dockerfile-oriented stubs or config strings tests check.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/41-containers-docker/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why multi-stage?
2. What does CGO_ENABLED=0 help?
3. Why nonroot?

Further reading

- https://docs.docker.com/language/golang/build-images/